

Evaluate-and-Repair for LLM-Generated NetOps Changes: A Preregistered Generate–Verify–Repair Study with a Deterministic Net Verifier

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We report a fully preregistered, deterministically archived, within-task paired experiment (N=300 canonical device-change tasks: routing 150 / ACL 100 / BGP 50) comparing (a) LLM-only proposals and (b) a Generate–Verify–Repair (GVR) pipeline: an LLM proposer (gpt-5-mini) whose proposals are gated by a frozen deterministic network verifier and, on rejection, given one automated repair iteration. The preregistered primary estimand is the paired reduction in verifier-visible unsafe proposals ("would-cause-unsafe-state-if-applied"). Results: complete pairs = 300; baseline unsafe = 1/300 (0.3333%); guarded (GVR) unsafe = 0/300 (0.00%); paired absolute reduction = 0.003333 (0.3333 percentage points); McNemar exact two-sided $p = 1.00$; bootstrap 95% percentile CI (10k, seed=1729) for absolute reduction = [0.00, 0.01]. The single baseline unsafe episode (task-000165) was corrected by the automated repair iteration; raw proposer outputs, verifier counterexamples, repair call JSON and all provenance are archived (execution_manifest: execution/raw_results.jsonl SHA-256 68c8168fff..., execution/transformation_manifest.jsonl SHA-256 f3ac5460...). We provide frozen verifier decision rules, the representative repaired episode with artifact citations, discussion of construct/external validity, and a minimal reproducibility recipe for deterministic replay.

I. INTRODUCTION

Generative LLMs can translate operator intent into device configuration proposals, but their stochastic outputs sometimes produce syntactically plausible yet operationally unsafe changes (e.g., unintended reachability, weakened ACLs, or non-reversible control-plane edits). Operational NetOps requires deterministic acceptance decisions and auditable evidence. The Generate–Verify–Repair (GVR) pattern interposes deterministic verifiers between probabilistic generation and acceptance and uses verifier counterexamples to guide repairs. Prior work advocates verifier-anchored workflows, and verification engines (Header-Space, SymNet, iptables semantics) provide deterministic predicates, yet there has been little preregistered, artifact-archived, within-task paired evaluation asking whether a frozen GVR pipeline measurably reduces verifier-visible unsafe proposals relative to LLM-only proposals. Research question (preregistered): Does a frozen GVR pipeline (gpt-5-mini proposer + deterministic verifier + ≤ 1 automated repair) reduce verifier-visible unsafe proposals relative to LLM-only proposals in a canonical NetOps sandbox? Contributions: (1) a fully preregistered experiment (CNSM2026-GVR-BATFISH-CAND-001; SHA-256 156e87e6...); (2) a frozen deterministic verifier with explicit decision rules archived before model calls (execu-

tion/transformation_manifest.jsonl SHA-256 f3ac5460...); (3) deterministic archival of every model call, verifier trace and repair call enabling replay (execution_manifest and file SHA-256s cited below); (4) confirmatory statistics and a representative repaired episode with archival pointers.

II. RELATED WORK

We position this study at the intersection of (A) Generate–Verify–Repair and verifier-guided generation architectures (see preregistered bibliography entries), (B) formal/static network verification (e.g., Header Space Analysis and SymNet/SEFL) that provide deterministic reachability primitives, and (C) surveys advocating assurance contracts and verifier gates for agentic NetOps. Representative verified sources used for method design and verifier predicates include SymNet (Stoenescu et al., SIGCOMM 2016 / DOI 10.1145/2934872.2934881), network update verification work (Li et al., COMNET 2020 / DOI 10.1016/j.comnet.2020.107326), and recent agentic NetOps surveys (arXiv:2605.12729). We used primary sources and archived the verification artifacts used to implement the deterministic predicates; references and the verification provenance records are included in the public execution manifest.

III. METHODOLOGY

Preregistration and frozen design. The experiment was preregistered as CNSM2026-GVR-BATFISH-CAND-001 (SHA-256 156e87e6...). The design frozen before any model calls specified: stratified deterministic sampling (seed 2026-08-25) producing 300 canonical device change tasks (routing 150, ACL 100, BGP 50); proposer model = gpt-5-mini; GVR flow: proposer $\rightarrow$ deterministic verifier gate $\rightarrow$ on rejection one automated repair invocation; maximum model calls = 600; primary outcome = paired binary indicator per task: verifier label "would-cause-unsafe-state-if-applied" (see explicit verifier checks below); primary test = McNemar exact two-sided; bootstrap 95% percentile CI (10,000 resamples, seed=1729). Execution provenance. Adapter family: hosted_netops_gvr_v1. Run completed 2026-08-25; completed episodes = 600; model calls used = 301. All raw provider calls, responses, parsed canonical patches, verifier counterexamples, repair prompts, repair responses and execution logs were archived and hashed (see execution_manifest; execution/raw_results.jsonl

SHA-256 68c8168fff..., execution/model_configuration.jsonl
SHA-256 a40e96e2..., execution/task_manifest.jsonl
SHA-256 7df39740a...). Deterministic frozen verifier (explicit decision rules). The primary binary outcome was implemented by a frozen deterministic verifier; its decision predicates were fixed and archived prior to model calls (execution/transformation_manifest.jsonl SHA-256 f3ac5460...). The verifier implements four deterministic checks (explicitly preregistered): 1) Reachability regression (header-space style): detect newly permitted packet tuples that the canonical policy labels as protected; counterexamples are recorded as packet tuples that now traverse previously blocked paths. 2) ACL/policy flip detection: detect rule transitions that change a protected flow from DENY→ALLOW or weaken ordering semantics to permit protected flows. 3) Parse / semantic invalidity: proposals that fail canonical parsing or contain semantic contradictions (e.g., overlapping prefixes producing unintended aggregation) are labeled unsafe. 4) Deterministic safety predicates: removal of mandated connectivity (for anchored services) or edits that remove required loopback/management interfaces, and edits that introduce non-reversible control-plane changes (e.g., deleting core routing instances) are unsafe. The verifier outputs structured counterexamples (packet tuple + violating rule) to feed the repair iteration. Scoring and exclusions. All scoring is deterministic and automated. Preplanned deterministic exclusions (e.g., proposals that could not be parsed were recorded as model failures) were applied automatically; paired analyses use only complete pairs. Contamination checks compared task/template hashes to pre-lock-development artifacts; flagged items would be excluded per the preregistration (none were excluded in this run). Statistical plan. Primary confirmatory test: McNemar exact two-sided on paired binary outcomes; report absolute and relative risk reductions and bootstrap percentile 95% CI (10k resamples, seed=1729). Power. The preregistration targeted a 50% relative reduction with N=300; further power calculations and sensitivity analysis details are archived with the preregistration.

IV. RESULTS

Aggregate confirmatory results. Complete paired episodes: 300. Paired 2×2 contingency counts (analysis/paired_contingency_table.csv SHA-256 3421adb5...): safe both = 299; unsafe baseline only = 1; unsafe guarded only = 0; unsafe both = 0. Observed rates: baseline unsafe = 1/300 = 0.003333 (0.3333%); guarded unsafe = 0/300 = 0.000000 (0.00%). Paired absolute reduction = 0.003333 (0.3333 percentage points). McNemar exact two-sided test (discordant b=1, c=0): p = 1.00. Bootstrap 95% percentile CI for absolute reduction (10,000 resamples, seed=1729): [0.00, 0.01] (i.e., [0.0%, 1.0%]). Task-type breakdown of observed unsafe proposals: routing (150): baseline 0 / guarded 0; ACL (100): baseline 1 / guarded 0; BGP (50): baseline 0 / guarded 0. Representative repaired episode (archival proof). Task ID: task-000165. Artifacts (archived with their

SHA-256s): execution/responses/task-000165-baseline.txt = 4c4df2ec851732cd7a6302211bbc79b5099a1bf6d470204b6ac883446bcd09
execution/responses/task-000165-guarded.txt = b4faabeaaf3bd9e0fae78adc1bbb3b40ff36d95e95ac6796c2f0000c2af2b8ff;
execution/provider_calls/task-000165-guarded-repair.json = 93d0712d93248452202c8a1f0051ca4a619a1c835a9af1f828494aed1f033eb
Sequence (extracted deterministically from archived logs):
(A) Baseline proposer output included an ACL patch that the verifier flagged because a canonical protected flow (packet tuple: src 10.0.0.0/24 → dst 172.16.0.0/12, proto TCP/22) would become permitted; the verifier emitted the structured counterexample (packet tuple + offending rule).
(B) The repair invocation used that counterexample as repair context and returned an amended ACL patch that the verifier accepted. The baseline unsafe produced by the LLM-only proposer was thereby corrected by the single automated repair iteration. Provenance and reproducibility. All provider calls, raw outputs, parsed canonical patches, verifier traces and analysis logs are archived in the execution manifest (execution/raw_results.jsonl SHA-256 68c8168f..., execution/transformation_manifest.jsonl SHA-256 f3ac5460..., execution/model_configuration.jsonl SHA-256 a40e96e2..., execution/task_manifest.jsonl SHA-256 7df39740a..., analysis/paired_contingency_table.csv SHA-256 3421adb5..., analysis/analysis_log.jsonl SHA-256 a509c04a...). Minimal recipe to reproduce confirmatory table: the six files enumerated in the previous sentence plus the analysis script archived alongside analysis/analysis_log.jsonl.

V. DISCUSSION

Interpretation. In this canonical sandbox the observed verifier-visible hazard rate under LLM-only proposals was extremely low (1/300). The GVR pipeline (verifier gate + one automated repair) corrected the single verifier-detected baseline unsafe proposal. The McNemar p = 1.00 reflects the low discordant count (b=1, c=0); the bootstrap 95% CI [0.00, 0.01] bounds the absolute reduction. The practical takeaway is not that GVR is universally required in all deployments, but that (i) deterministic verifiers provide auditable failure signals and (ii) short repair loops can automatically resolve verifier-detectable hazards in this canonical setting. Threats to validity. Construct validity depends on verifier coverage: our frozen verifier covers deterministic reachability regression (header-space style), ACL/policy flips for protected flows, parse/semantic validity, and deterministic safety predicates for mandated connectivity and non-reversible control-plane edits; vendor-specific runtime, multi-change interactions, timing/race conditions and some hardware failover microbehaviors were intentionally out of scope (see Limitations). Internal validity mitigations: shared initial candidate semantics and deterministic seeding minimize prompt/model nondeterminism as a confound; all raw artifacts and provider calls were archived to permit replay. External validity. Single-model scope (gpt-5-mini), canonical task bank, and synthetic sandbox semantics limit claim breadth; production adoption requires multi-model evaluation, vendor-instrumented verification, canary deployments

and rollback-aware trials. Operational implications. A deterministic verifier gate yields: auditable counterexamples, deterministic acceptance decisions, and the ability to create convergent repair loops that convert stochastic proposals into verifier-acceptable artifacts. For production pipelines we recommend expanding verifier coverage to vendor specifics, combining GVR with canary/promote/rollback and human escalation policies, and instrumenting closed-loop monitoring for post-deployment verification.

VI. LIMITATIONS

- Single-model limitation: experiments used a single proposer (gpt-5-mini); results do not establish multi-model generality.
- Canonical task bank: tasks were deterministic canonical templates (routing, ACL, BGP); production change distributions may differ materially and have higher baseline failure rates.
- Verifier coverage: the deterministic verifier models a canonical subset (reachability regressions, ACL/policy flips, parse/semantic invalidity, mandated-connectivity removals) but omits some vendor-specific coupled behaviors and multi-change interactions (verifier implementation and decision rules archived: execution/transformation_manifest.json SHA-256 f3ac5460...).
- Low baseline failure prevalence: only 1/300 baseline unsafe proposals were observed, limiting statistical power and widening uncertainty for modest absolute reductions.
- Sandbox abstraction: the canonical sandbox is a model of device semantics for deterministic checking and is not a substitute for vendor device execution; production deployment requires extended validation on vendor devices and in live canary trials.

VII. CONCLUSION

We presented a fully preregistered, deterministically archived within-task paired evaluation of a GVR pipeline for LLM-generated NetOps changes. Observed verifier-visible unsafe proposals were 1/300 for LLM-only vs 0/300 for the guarded GVR pipeline; paired absolute reduction = 0.003333 (0.3333 percentage points); McNemar exact two-sided $p = 1.00$; bootstrap 95% CI = [0.00, 0.01]. The single baseline unsafe instance (task-000165) was repaired by one automated iteration; artifacts and provenance are archived for deterministic replay. The results show that, in this canonical sandbox, a frozen deterministic verifier plus a short repair loop can detect and correct verifier-visible hazards; broader operational conclusions require larger, multi-model, vendor-instrumented studies. All required artifacts and a minimal replay recipe are provided in the disclosure to support independent audit and follow-on research.

DISCLOSURE STATEMENT

Mandatory disclosure (concise archival pointers). Human role and autonomy boundary: the human Research Director selected the topic family (Generative AI / LLMs for NetOps)

and approved pre-lock infrastructure development. After the autonomy lock the autonomous system performed literature discovery, research-gap selection, preregistration, experiment design, code generation, execution, deterministic scoring, statistical analysis, autonomous peer review, revision, and manuscript drafting without manual technical edits. Models and orchestration: proposer model = gpt-5-mini (hosted API); adapter family = hosted_netops_gvr_v1; all provider calls and raw outputs were archived. Exact preregistration: CNSM2026-GVR-BATFISH-CAND-001 (SHA-256 156e87e6ce4a9a2348525fa8960f35fb4120031344cf4b4a38b247b10893181). Verifier and frozen rules: deterministic verifier implementation and decision rules were fixed and archived before any model calls (execution/transformation_manifest.json; SHA-256 f3ac546043ac61cb7222b01c975a3cb98d8f309bb7f65f636ab98667e98608b). Execution archive (minimal reproduction pointers with SHA-256): execution/raw_results.jsonl : 68c8168fff763b5530f4b36681a8956f5bbfb4ff98f6753a12cae97b017ee8e1; execution/transformation_manifest.json (verifier & decision rules) : f3ac546043ac61cb7222b01c975a3cb98d8f309bb7f65f636ab98667e98608b; execution/model_configuration.json : a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd82; execution/task_manifest.jsonl : 7df39740a15b3d8b2ff7deb49f99ff6b2ee46540b41af2ef0562137de0e550bb; analysis/paired_contingency_table.csv : 3421adb52d5791a979397368165fc2062e6b9a4a771ce49c0bcac80b02215d; analysis/analysis_log.jsonl : a509c04a5f75eee720ad01e443807b72d1c670a. Representative repaired episode: task-000165 artifacts: execution/responses/task-000165-baseline.txt = 4c4df2ec851732cd7a6302211bbc79b5099a1bf6d470204b6ac883446bcd09; execution/responses/task-000165-guarded.txt = b4faabeaaf3bd9e0fae78adc1bbb3b40ff36d95e95ac6796c2f0000c2af2b8ff; execution/provider_calls/task-000165-guarded-repair.json = 93d0712d93248452202c8a1f0051ca4a619a1c835a9af1f828494aed1f033eb. Provider endpoints/credentials are not published. Execution integrity: planned episodes = 600; completed = 600; model calls used = 301; no provider or infrastructure failures affected scientific outcomes. Pre-lock development: prior infrastructure rehearsals occurred but the confirmatory experiment used only artifacts and design frozen in the preregistration; contamination checks were applied and logged. No human manually edited experiment outputs, analysis results, or technical manuscript content after the autonomy lock. Reproducibility minimal recipe: to reproduce the confirmatory table/figure, the minimal required artifacts are the archived files listed above (execution/raw_results.jsonl; execution/transformation_manifest.json; execution/model_configuration.json; execution/task_manifest.jsonl; analysis/paired_contingency_table.csv; analysis/analysis_log.jsonl) plus the archived analysis scripts referenced in the execution_manifest. Limitations and threats to validity are stated in the manuscript.

REFERENCES

- [1] doi:10.1145/2934872.2934881

- [2] <https://doi.org/10.1016/j.comnet.2020.107326>
- [3] <https://arxiv.org/abs/2605.12729>
- [4] Rafael Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", 2026, 10.2139/ssrn.6754899
- [5] <http://arxiv.org/abs/2312.10997>
- [6] CNSM2026-GVR-BATFISH-CAND-001